Complemento de la propuesta de PulsoCheck

Estado de este documento: actualizado el 9 de noviembre de 2026 con base en el código del repositorio. Describe las funciones implementadas y señala por separado las validaciones aún pendientes en dispositivos físicos. La compilación final y el comportamiento en cada modelo de reloj deben comprobarse antes de declarar el proyecto terminado.

1. Arquitectura

PulsoCheck se organiza como una aplicación Android con dos módulos y capas internas. El módulo **wear** contiene la experiencia del reloj y **mobile** la experiencia del teléfono. Cada módulo tiene su propia interfaz y almacenamiento local. Los modelos equivalentes se convierten en la capa de datos al intercambiarse mediante Wearable Data Layer.

Dentro de cada módulo se separarán estas responsabilidades:

1. **Presentación:** pantallas de Compose y estado visible para la persona. Las pantallas no acceden directamente al sensor ni a la base de datos.
2. **Dominio:** reglas del proyecto, como validar lecturas y decidir si forman un episodio. Se mantienen independientes de Android para que puedan entenderse y verificarse con datos de ejemplo.
3. **Datos:** repositorios que coordinan Room, Health Services y Wearable Data Layer. Esta capa convierte la información externa a modelos propios de PulsoCheck.

La presentación utiliza **MVVM**: una pantalla Compose observa el estado de un **ViewModel** y le comunica acciones del usuario. Por ejemplo, el botón de seguimiento llama una función del **ViewModel**; no modifica el sensor directamente.

Fundamentación

El proyecto tiene dos interfaces y varias fuentes de datos (sensor, base local y conexión entre dispositivos). Separarlas reduce el acoplamiento: un cambio en la interfaz no obliga a reescribir las reglas, y una sustitución de almacenamiento no debe cambiar las pantallas. Mantener los módulos **wear** y **mobile** separados también permite desarrollar e instalar cada aplicación de acuerdo con las capacidades de su dispositivo.

La arquitectura se mantiene deliberadamente sencilla. En el módulo **wear**, **TrackingViewModel** coordina Health Services, Room, la detección de episodios y Wearable Data Layer. En **mobile**, **MobileHomeViewModel** observa los repositorios y expone lecturas, episodios y respuestas a Compose. Las implementaciones Room quedan detrás de contratos de repositorio.

2. Patrones de diseño

Los patrones aplicados en el código se describen a continuación:

Patrón	Aplicación	Motivo
MVVM	**ViewModel** entre Compose y las acciones/estado de pantalla.	Mantener la interfaz enfocada en mostrar datos y recibir interacciones.
Repository	Repositorios para lecturas, episodios, respuestas y sincronización.	Evitar que las reglas y pantallas dependan directamente de Room o de un servicio de Google.
Observer / flujo reactivo	**Flow** de Room y estado observable en el **ViewModel**.	Actualizar el historial cuando se guarden datos, sin que la pantalla tenga que consultar repetidamente.
Adapter	Convertidores entre eventos de Health Services, mensajes de Data Layer y modelos de PulsoCheck.	Aislar formatos externos y mantener un modelo interno estable.
State	**TrackingStatus** representa estados del seguimiento; estados observables en los ViewModel reflejan datos y operación de borrado.	Hacer explícito qué está ocurriendo y permitir que la interfaz reaccione a cambios.

Los patrones anteriores están presentes en el código en distintos grados. La sincronización implementa entrega mediante Data Layer y limpieza de elementos procesados; no existe un protocolo de confirmación de recepción con acuse explícito

del reloj. Las pruebas de integración entre dispositivos siguen pendientes.

3. Funciones implementadas

La tabla resume las funciones que ya aparecen en el código y su responsabilidad principal. Los nombres exactos pueden variar por capa; se mencionan clases y métodos representativos para facilitar su localización.

Área	Funciones implementadas	Responsabilidad
Seguimiento	**iniciarSeguimiento()**, **detenerSeguimiento()**, **admiteFrecuenciaCardiaca()**, **observarFrecuenciaCardiaca()**	Consultar capacidad y permiso, registrar el callback de MeasureClient mientras se solicita seguimiento y cancelar la escucha al detenerla.
Lecturas	**guardar()** en repositorios Room; **enviar()** en **HeartRateDataLayerSender**	Conservar lecturas identificables en el reloj y transmitirlas al teléfono. El receptor valida campos y antigüedad antes de guardarlas.
Episodios	**DetectHeartRateEpisodesUseCase.invoke()**	Abrir episodio tras tres lecturas consecutivas sobre el umbral y cerrarlo tras dos lecturas consecutivas dentro de este, respetando la separación máxima configurada. No emite diagnósticos.
Respuesta	**registrarRespuesta()**	Guardar localmente la opción de bienestar y hora, y enviar la respuesta al teléfono.
Sincronización	**enviar()**, **enviarEpisodio()**, **enviarRespuesta()** y **onDataChanged()** de los listeners	Transferir lecturas, episodios y respuestas mediante elementos identificables de Data Layer. Los listeners validan, guardan en Room y solicitan limpiar el elemento recibido.
Historial	**observarDesde()**, **observarTodas()**, **DailyReadingsCard**	Mostrar episodios, respuestas, lecturas y resúmenes diarios de los últimos cinco días del teléfono; los días sin lecturas aparecen como “Sin datos”.
Avisos y privacidad	**notificarEpisodio()**, **borrarHistorialCompleto()**	Notificar episodios recientes según permisos. El borrado del teléfono elimina sus lecturas, episodios y respuestas en una transacción; no borra la copia guardada en el reloj.

La regla implementada considera tres lecturas válidas consecutivas por encima del límite para abrir un episodio y dos lecturas consecutivas dentro del límite para cerrarlo. Un registro inválido o una separación superior al máximo reinicia los conteos consecutivos. Estos parámetros son de software y no criterios clínicos; el umbral y la separación máxima están definidos en la configuración del módulo Wear.

4. Intercambio de datos entre reloj y teléfono

La aplicación utiliza **Wearable Data Layer** para comunicarse entre los módulos instalados. No implementa su propio emparejamiento Bluetooth: la vinculación reloj-teléfono queda a cargo de Google Pixel Watch y Android. Data Layer puede aprovechar Bluetooth y, cuando esté disponible, otra conexión de red.

- **DataClient:** utilizado con rutas que incorporan identificadores para lecturas, episodios y respuestas. Las solicitudes se marcan como urgentes.
- **MessageClient:** no se utiliza en esta versión; las funciones implementadas no dependen de una llamada efímera de mensaje.
- **Identificador único:** cada lectura, episodio y respuesta tendrá un identificador estable. Al recibir un registro repetido, el almacenamiento lo reconocerá para no duplicarlo.
- **Recepción y limpieza:** el teléfono valida y guarda en Room y después solicita eliminar el elemento transferido del Data Layer. El reloj mantiene la copia de negocio en Room. No hay un acuse explícito de vuelta que marque cada ID como sincronizado.
- **Datos atrasados:** se agregarán al historial, pero no producirán una notificación que parezca indicar un episodio actual.

El reloj guarda localmente lecturas, episodios y respuestas, por lo que estas operaciones no requieren que el teléfono esté conectado. El envío al teléfono depende de que Wearable Data Layer pueda sincronizar.

5. Notificaciones, cámara y sensores

Notificaciones y vibración

El reloj muestra una notificación local para un episodio reciente y permite registrar la respuesta desde la aplicación del propio reloj. Al tocar el aviso se abre PulsoCheck. La vibración dependerá de las capacidades, permisos, modo No molestar y ajustes de notificación del dispositivo. El teléfono muestra un aviso informativo al recibir un episodio reciente y abre la aplicación para revisar el registro; la respuesta se realiza en el reloj.

Cámara

La cámara **no se utilizará** en la primera versión. La propuesta no define una función que la necesite: la respuesta se elige con controles táctiles y el pulso procede del sensor del reloj. Excluir cámara evita pedir un permiso sin justificación y reduce la recolección de datos personales.

Sensor de frecuencia cardiaca

En el reloj se consulta la capacidad **HEART_RATE_BPM** de **Health Services** y se solicita el permiso de lectura correspondiente. El código utiliza **MeasureClient** mientras se inicia el seguimiento desde la aplicación. **No se implementó PassiveMonitoringClient ni una promesa de lectura continua en segundo plano.** El usuario debe validar la disponibilidad y duración de las lecturas en su reloj físico.

Health Services puede limitar u organizar las lecturas de acuerdo con las capacidades del dispositivo y el estado de la aplicación. La aplicación muestra la hora de medición y no promete intervalos exactos ni vigilancia continua. No se provocarán elevaciones de pulso para probar el sistema.

6. Base de datos

Room sobre SQLite almacena los datos localmente en cada dispositivo, para que el reloj pueda registrar un episodio y una respuesta sin conexión. El esquema implementado incluye:

- **Lectura:** identificador, valor en lpm, fecha/hora y origen. No hay una columna persistente de estado de sincronización.
- **Episodio:** identificador, inicio, fin, lecturas asociadas y estado de respuesta.
- **Respuesta:** identificador del episodio, opción elegida y fecha/hora de guardado.
- **Configuración:** el umbral y la separación máxima del detector se suministran desde **HeartRateMonitoringConfig**; todavía no hay una pantalla de ajustes persistentes.

Los DAO de Room contienen operaciones de consulta, guardado y eliminación. El teléfono conserva la ventana reciente de cinco días y presenta resúmenes con datos existentes. Un día sin lecturas se marca **Sin datos**; no se inventan valores. La interfaz ofrece borrar las lecturas, episodios y respuestas del teléfono con confirmación y transacción. El reloj conserva sus propios registros; el borrado entre dispositivos no está implementado.

7. APIs internas y externas

En este proyecto, "API" puede referirse a funciones de Android/Google usadas dentro de la aplicación; no implica que exista un servidor propio.

API o servicio	Tipo	Uso
Health Services (**MeasureClient**)	API de plataforma/Google para Wear OS	Consultar capacidades y obtener eventos de frecuencia cardiaca mientras la app inicia el seguimiento. No se usa **PassiveMonitoringClient**.
Wearable Data Layer (**DataClient**)	API de Google Play services	Intercambiar lecturas, episodios y respuestas entre reloj y teléfono. **MessageClient** no se usa.
Room / SQLite	Biblioteca Android y base de datos local	Persistir registros y observar cambios localmente.
Notificaciones Android	API del sistema Android	Crear avisos locales y dirigir al detalle/formulario correcto.

API o servicio	Tipo	Uso
Sensores de cámara	No se usan	No existe una necesidad de cámara dentro del alcance definido.
Servidor o API web propia	No se contempla en la primera versión	No habrá cuentas, nube ni transmisión a servicios externos propios.

8. Versiones y buenas prácticas

El código fuente está alojado en el repositorio privado de GitHub **EmanuelC1601/PulsoCheck**. Se trabaja con commits descriptivos y revisiones de **git status** y **git diff**. El equipo informó que invitó a la asesora como colaboradora; su aceptación debe confirmarse en GitHub.

No se incluirán en Git **local.properties**, claves de firma, contraseñas, archivos de compilación ni datos reales de salud. Los cambios se explicarán antes de integrarlos. El código nuevo mantendrá nombres descriptivos y comentarios útiles para explicar decisiones, evitando comentarios que solo repitan la instrucción de una línea.

9. Conclusiones

La propuesta está implementada como dos módulos Android, **wear** y **mobile**, con arquitectura por capas y MVVM en presentación. Repositories, flujos observables, adaptadores y estados explícitos separan interfaz, reglas, almacenamiento, sensor y comunicación. Health Services entrega lecturas que estén disponibles; Room conserva datos localmente; Wearable Data Layer transfiere registros cuando el sistema puede sincronizar.

La primera versión no usa cámara, nube propia, diagnóstico ni recomendaciones de tratamiento. Las notificaciones dependen de permisos y ajustes del dispositivo. Las lecturas atrasadas se conservan como historial y no se presentan como un episodio actual. La compilación final y las pruebas de integración en el reloj y teléfono físicos son criterios de aceptación pendientes; las advertencias se deben registrar aparte de los errores.

10. Relación con la propuesta inicial

Este complemento amplía el documento inicial de PulsoCheck. Los requisitos de usuario, sensores, funcionamiento sin conexión, regla de episodios, historial, almacenamiento y validación se mantienen como fuente de alcance. Las funciones indicadas como implementadas deben validarse en el dispositivo objetivo antes de considerarlas verificadas.